Task 2- : Automating EC2 Start/Stop with Jenkins Pipelines (Tag-Based)

1. Provision EC2 Instances with Tags

We first create two EC2 instances with the appropriate tags:

- Instance 1 → `env=dev`
- Instance 2 → `env=prod`

Using AWS CLI:

```
# Create Dev EC2
aws ec2 run-instances \
  --image-id ami-0c02fb55956c7d316 \
  --instance-type t2.micro \
  --key-name my-key \
  --security-group-ids sg-xxxxxx \
  --subnet-id subnet-xxxxxx \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=env,Value=dev}]'

# Create Prod EC2
aws ec2 run-instances \
  --image-id ami-0c02fb55956c7d316 \
  --instance-type t2.micro \
  --key-name my-key \
  --security-group-ids sg-xxxxxx \
  --subnet-id subnet-xxxxxx \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=env,Value=prod}]'
```

---

2. Jenkins Setup

- Install Jenkins on a separate EC2 (Amazon Linux/Ubuntu).
- Install plugins:
  - **Pipeline**
  - **AWS CLI** (or configure AWS CLI manually inside Jenkins server).
- Configure AWS credentials in Jenkins:
  - Either attach an IAM role.
  - Or configure `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` via Jenkins Credentials Manager.

Verify AWS CLI works:

```
aws ec2 describe-instances --region us-east-1
```

---

3. Create Jenkins Pipelines

Pipeline 1: Start Dev EC2

```
pipeline {
    agent any
    stages {
        stage('Start Dev EC2') {
            steps {
                sh '''
                INSTANCE_ID=$(aws ec2 describe-instances \
                  --filters "Name=tag:env,Values=dev" \
                  --query "Reservations[*].Instances[*].InstanceId" \
                  --output text)

                echo "Starting Dev EC2: $INSTANCE_ID"

                aws ec2 start-instances --instance-ids $INSTANCE_ID
                aws ec2 wait instance-running --instance-ids $INSTANCE_ID
                echo "✅ Dev EC2 started successfully"
                '''
            }
        }
    }
}

```

Pipeline 2: Stop Dev EC2

```
pipeline {
    agent any
    stages {
        stage('Stop Dev EC2') {
            steps {
                sh '''
                INSTANCE_ID=$(aws ec2 describe-instances \
                  --filters "Name=tag:env,Values=dev" \
                  --query "Reservations[*].Instances[*].InstanceId" \
                  --output text)

                echo "Stopping Dev EC2: $INSTANCE_ID"

                aws ec2 stop-instances --instance-ids $INSTANCE_ID
                aws ec2 wait instance-stopped --instance-ids $INSTANCE_ID
                echo "✅ Dev EC2 stopped successfully"
                '''
            }
        }
    }
}

```

---

4. Run Jenkins Jobs

- Create two jobs in Jenkins:
  - `Start-Dev-EC2`
  - `Stop-Dev-EC2`
- Run `Start-Dev-EC2`:
  - Jenkins console should log instance ID.
  - EC2 console → instance status = **running**.

- Run `Stop-Dev-EC2`:
  - Jenkins console should log instance ID.
  - EC2 console → instance status = **stopped**.

---

5. IAM Permissions

Attach an IAM policy with least privileges to the Jenkins server role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:StartInstances",
        "ec2:StopInstances"
      ],
      "Resource": "*"
    }
  ]
}

```

Final Outcome

- Two EC2 instances provisioned (`dev`, `prod`).
- Jenkins pipelines created for:
  - Starting **Dev EC2**.
  - Stopping **Dev EC2**.
- Verified logs:
  - Jenkins pipeline logs show instance ID + status changes.
  - AWS Console shows state transition (`stopped → running` and vice versa).
- IAM policy ensures least privilege for Jenkins server.

Expected Screenshots to Submit

- Jenkins job execution logs.
- AWS EC2 console showing instance state change.
- IAM policy attached to Jenkins server role.

Reference: [Jenkins Tutorial for Beginners – Full Guide to CI/CD]

Bonus :

Try creating an additional Jenkins pipeline to start and stop the **ECS Fargate service** created in Task 1.

The below link is an reference using Lamda service ,the bonus task is to be performed with **Jenkins pipeline** instead for Lambda .

[Automatically Stop ECS Fargate Services with AWS Lambda and EventBridge | by Darnel Edwards | Medium]